SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – TRANSFORMER MECHANICS WORKSHEET

Course Code:	CSA6506	**Course Title:**	Generative AI and Large Language Models
CO Assessed:	CO1 – Precision (BL1, BL2)	**Assessment:**	Assessment Tool 3 – Transformer Mechanics Worksheet
Weightage:	40% of CIA	**Total Marks:**	100
CO1 Statement:	*Explain the fundamentals of Artificial Intelligence, Generative AI, Foundation Models, Transformer architecture, tokenization, embeddings, and Large Language Models. (BL1, BL2)*		
Student Name:	N.Hemanth Reddy	**Reg. No.:**	192472215
Section / Batch:	CSE-AI	**Date:**	

Instructions to Students

• Answer all questions. It carries 100 Marks. Total: 100 Marks.

· Tokenize a sentence.
· Convert tokens to token IDs.
· Explain embeddings.
· Label a Transformer encoder.
· Identify Query, Key, and Value.
· Explain which words receive higher attention.
· Compare Transformer models.
· Complete a Transformer pipeline.
· Draw a block diagram.

Code of Conduct

I N.Hemanth Reddy (192472215) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____N.Hemanth Reddy_____

TRANSFORMER MECHANICS WORKSHEET

Question 1: Sentence Tokenization

Consider the sentence: “Transformers enable intelligent text generation.”

Split the sentence into tokens using a suitable tokenizer. Explain how the tokenization may differ between a word-level tokenizer and a subword tokenizer.

Topic: Tokenization | Bloom’s level: Understand

Question 2: Token-to-ID Conversion

Given the token sequence `["AI", "models", "learn"]` and the vocabulary mapping `{"AI": 12, "models": 25, "learn": 8}`, convert the tokens into their corresponding token IDs. What happens if a token is not present in the vocabulary?

Topic: Token IDs | Bloom’s level: Apply

Question 3: Understanding Embeddings

Explain how token IDs are converted into embedding vectors in a Transformer. Why are embedding vectors more useful for language processing than token IDs alone?

Topic: Token embeddings | Bloom’s level: Understand

Question 4: Transformer Encoder Labelling

Draw a Transformer encoder block and label the following components in the correct order: multi-head self-attention, input embeddings with positional information, feed-forward neural network, residual connections, and layer normalization.

Topic: Transformer encoder architecture | Bloom’s level: Apply

Question 5: Query, Key, and Value Identification

In the sentence “The student submitted the assignment because it was complete,” explain how Query, Key, and Value vectors are generated from token representations. Describe their respective roles in determining relationships between words.

Topic: Query–Key–Value | Bloom’s level: Analyze

Question 6: Attention Weight Analysis

Consider the sentence: “The cat chased the mouse because it moved quickly.” Which words might receive higher attention when processing the word “it”? Justify your answer and explain how attention weights help a Transformer capture contextual relationships.

Topic: Self-attention | Bloom’s level: Analyze

Question 7: Comparing Transformer Models

Compare BERT, GPT, and T5 in terms of their attention approach, input-output behaviour, and typical applications. Identify which model would be most suitable for (a) text classification, (b) open-ended text generation, and (c) text summarization, with justification.

Topic: Transformer model comparison | Bloom's level: Analyze

Question 8: Completing the Transformer Pipeline

Complete the following Transformer processing pipeline by filling in the missing stages:

Text Input → _____ → _____ → Embedding and Positional Information → _____ → Output Probabilities → Predicted Token.

Explain the function of each missing stage.

Topic: Transformer pipeline | Bloom's level: Apply

Question 9: Transformer Block Diagram

Draw a block diagram illustrating how an input sentence is processed by a Transformer-based language model to generate an output. Include tokenization, token ID conversion, embeddings, attention layers, feed-forward layers, output projection, and token prediction.

Topic: Transformer architecture | Bloom's level: Create

Question 10: Integrated Transformer Mechanics

A language model receives the sentence "Generative AI creates new content." Trace the sentence through tokenization, token ID conversion, embedding generation, self-attention, contextual representation, and output prediction. Explain how the attention mechanism helps the model understand relationships among the words.

Topic: Integrated Transformer pipeline | Bloom's level: Analyze

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Mechanics Explanation	30%	All key concepts (AI, ML, DL, GenAI, LLMs) correctly identified with clear hierarchical relationships	Most concepts correct; minor gaps in relationships	Several concepts missing or misclassified	Major conceptual errors; map incomplete
Worked-Example Accuracy	25%	Real-world examples linked accurately to each concept	Examples mostly relevant	Few or generic examples	No real-world linkage shown
Mathematical Reasoning	20%	Logical, well-organized structure with clear connectors	Mostly organized; minor structural issues	Some disorganization affecting clarity	Poorly structured, hard to follow
Diagram & Working Clarity	15%	Visually clear, creative, easy to interpret	Neat and readable	Cluttered or inconsistent formatting	Untidy, difficult to interpret
Step Justification	10%	Concise labels/notes that add clarity	Adequate labeling	Minimal or unclear labeling	No supporting labels or explanation

Marks Evaluation:

Mechanics Explanation (30%)	Worked-Example Accuracy(25%)	Mathematical Reasoning (20%)	Diagram & Working Clarity (15%)	Step Justification (10%)	Total Marks (100)

SOLUTION:

TRANSFORMER MECHANICS WORKSHEET

Question 1: Sentence Tokenization

Given Sentence: “Transformers enable intelligent text generation.”

Answer:

Tokenization is the process of dividing a sentence into smaller units called tokens.

1. Word-Level Tokenization:

The sentence is divided into individual words and punctuation marks.

Tokens: ["Transformers", "enable", "intelligent", "text", "generation", "."]

2. Subword Tokenization:

Subword tokenization divides words into smaller units when necessary. For example, a tokenizer may split "Transformers" into ["Transform", "ers"]. The exact tokens depend on the tokenizer.

Difference:

1. Word-level tokenization treats each complete word as a token.
2. Subword tokenization divides some words into smaller units.
3. Subword tokenization helps handle rare and unfamiliar words.

Conclusion: Tokenization converts text into smaller units that can be processed by a language model.

Question 2: Token-to-ID Conversion

Given:

Token sequence: ["AI", "models", "learn"]

Vocabulary mapping:

- AI = 12
- models = 25
- learn = 8

Conversion:

AI → 12

models → 25

learn → 8

Final Token ID Sequence: [12, 25, 8]

What happens if a token is not present in the vocabulary?

If a token is not present in the vocabulary, the tokenizer may divide it into known subword tokens. If this is not possible, it may use a special unknown token called [UNK].

Conclusion: Token IDs represent tokens numerically and allow the model to process text.

Question 3: Understanding Embeddings

Answer:

Token embeddings convert token IDs into numerical vectors that represent information about tokens.

Working:

1. The input text is divided into tokens.
2. Each token is assigned a unique token ID.
3. The embedding layer converts each token ID into a numerical vector.
4. Positional information is added to represent the order of tokens.
5. The vectors are passed to the Transformer layers.

Example:

Suppose the token ID of "AI" is 12. Its embedding may be represented as:

E(12) = [0.25, 0.61, -0.18, 0.42]

These values are illustrative; actual embeddings usually have many dimensions.

Advantages of Embeddings:

1. Token IDs are only numerical labels and do not directly represent meaning.
2. Embeddings provide useful numerical representations for language processing.
3. They help the model learn semantic and syntactic relationships.
4. They enable attention and neural network operations.

Conclusion: Embeddings convert token IDs into meaningful numerical representations that help Transformers process language.

Question 4: Transformer Encoder Labelling

Answer:

A Transformer encoder processes input embeddings using self-attention and feed-forward neural networks. Residual connections and layer normalization help stabilize the processing.

Block Diagram:

Input Embeddings + Positional Information

|

v

Multi-Head Self-Attention

|

v

Residual Connection + Layer Normalization

|

v

Feed-Forward Neural Network

|

v

Residual Connection + Layer Normalization

|

v

Contextual Output Representations

Explanation:

1. Input Embeddings: Convert tokens into numerical vectors.
2. Positional Information: Represents the order of tokens.
3. Multi-Head Self-Attention: Identifies relationships between different tokens.
4. Residual Connections: Add the input of a sublayer to its output.
5. Layer Normalization: Helps maintain stable neural network processing.
6. Feed-Forward Neural Network: Applies learned transformations to token representations.

Conclusion: The Transformer encoder captures contextual relationships between tokens using self-attention and feed-forward layers.

Question 5: Query, Key, and Value Identification

Given Sentence: "The student submitted the assignment because it was complete."

Answer:

Query (Q), Key (K), and Value (V) are vectors generated from token representations using learned weight matrices.

$Q = XW_Q$

$K = XW_K$

$V = XW_V$

Where:

X = Input token representations

W_Q = Query weight matrix

W_K = Key weight matrix

W_V = Value weight matrix

Roles of Query, Key, and Value:

1. Query (Q): Represents the information a token is looking for.
2. Key (K): Represents the information a token provides for matching.
3. Value (V): Contains the information used to calculate the attention output.

Example:

In the sentence, the word "it" may refer to "the assignment." The model examines relationships between

words to determine which tokens are relevant to understanding the sentence.

The attention mechanism calculates similarity scores between queries and keys, converts them into weights using softmax, and combines the value vectors.

$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{dk})V$

Here, dk is the dimension of the key vectors.

Conclusion: Queries search for relevant information, keys determine relevance, and values provide information for generating contextual representations.

Question 6: Attention Weight Analysis

Given Sentence: "The cat chased the mouse because it moved quickly."

Answer:

When processing the word "it," a Transformer may assign higher attention weights to words that help identify its reference.

Possible Attention Relationships:

1. Mouse: A likely reference because the mouse may have moved quickly.
2. Cat: Another possible reference depending on the context.
3. Moved: Helps identify the action associated with "it."
4. Quickly: Provides information about the manner of movement.

The word "mouse" is a plausible candidate, although the sentence alone does not establish the reference with complete certainty.

How Attention Weights Work:

1. The model calculates similarity scores between the query for "it" and the keys of other tokens.
2. Softmax converts these scores into attention weights.
3. Higher weights allow more information from the corresponding value vectors to contribute to the output.
4. The model combines this information to produce a contextual representation.

Conclusion: Self-attention helps Transformers identify relationships between words, understand context, and resolve possible pronoun references.

Question 7: Comparing Transformer Models

Answer:

BERT, GPT, and T5 are Transformer-based models, but they differ in architecture, attention mechanisms, and applications.

1. BERT:

- Architecture: Encoder-only.
- Attention: Bidirectional self-attention.

- Input-Output Behaviour: Produces contextual representations for language understanding tasks.
- Applications: Text classification, sentiment analysis, and named entity recognition.

2. GPT:

- Architecture: Decoder-only.
- Attention: Causal self-attention.
- Input-Output Behaviour: Predicts the next token to generate text.
- Applications: Text generation, chatbots, and code generation.

3. T5:

- Architecture: Encoder-decoder.
- Attention: Bidirectional attention in the encoder and causal attention in the decoder.
- Input-Output Behaviour: Converts input text into output text.
- Applications: Text summarization, translation, and other text-to-text tasks.

Suitable Models for Applications:

(a) Text Classification – BERT

BERT is suitable because bidirectional attention helps it understand the context of words from both directions.

(b) Open-Ended Text Generation – GPT

GPT is suitable because it predicts the next token based on previous tokens and generates text sequentially.

(c) Text Summarization – T5

T5 is suitable because its encoder-decoder architecture processes the input text and generates a shorter summary.

Conclusion: BERT is commonly used for language understanding, GPT for text generation, and T5 for text-to-text tasks.

Question 8: Completing the Transformer Pipeline

Given Pipeline:

Text Input → _____ → _____ → Embedding and Positional Information → _____ → Output Probabilities → Predicted Token

Completed Pipeline:

Text Input → Tokenization → Token ID Conversion → Embedding and Positional Information → Transformer Layers → Output Probabilities → Predicted Token

Functions of Each Stage:

1. Tokenization: Divides input text into smaller tokens.
2. Token ID Conversion: Assigns numerical IDs to the tokens.
3. Embedding and Positional Information: Converts token IDs into vectors and represents token

order.

4. Transformer Layers: Apply attention and feed-forward operations to process contextual information.
5. Output Probabilities: Convert output scores into probabilities for possible tokens.
6. Predicted Token: Selects the next token using a decoding strategy.

Example:

For the input "AI creates," the model may predict "content" as the next token.

Conclusion: The Transformer pipeline converts text into numerical representations, processes context, and predicts output tokens.

Question 9: Transformer Block Diagram

Answer:

A Transformer-based language model processes input text through tokenization, numerical representation, attention layers, and output prediction.

Block Diagram:

Input Sentence

|

v

Tokenization

|

v

Token ID Conversion

|

v

Embedding + Positional Information

|

v

Self-Attention Layers

|

v

Feed-Forward Neural Networks

|

v

Output Projection (Logits)

|

v

Output Probabilities

|

v

Token Prediction

|

v

Generated Text

Explanation:

1. Input Sentence: Receives text from the user.
2. Tokenization: Divides the sentence into tokens.
3. Token ID Conversion: Assigns numerical IDs to the tokens.
4. Embeddings: Convert token IDs into vectors and incorporate positional information.
5. Self-Attention Layers: Identify relationships between tokens.
6. Feed-Forward Neural Networks: Process contextual representations.
7. Output Projection: Converts the final representations into vocabulary scores.
8. Output Probabilities: Assign probabilities to possible next tokens.
9. Token Prediction: Selects the next token and repeats the process when more text is required.

Conclusion: The Transformer architecture uses attention and neural network layers to process context and generate appropriate text.

Question 10: Integrated Transformer Mechanics

Given Sentence: "Generative AI creates new content."

Answer:

The sentence passes through tokenization, token ID conversion, embedding generation, self-attention, contextual representation, and output prediction.

Step 1: Tokenization

The sentence is divided into tokens.

Example:

["Generative", "AI", "creates", "new", "content", "."]

The exact tokens depend on the tokenizer used.

Step 2: Token ID Conversion

Each token is mapped to a numerical ID using the model's vocabulary.

Illustrative mapping:

Generative → 101

AI → 205

creates → 317

new → 408

content → 512

. → 9

The resulting illustrative sequence is:

[101, 205, 317, 408, 512, 9]

These IDs are examples only; actual token IDs depend on the model.

Step 3: Embedding Generation

Each token ID is converted into a learned numerical vector. Positional information is included to represent token order.

Example:

E(AI) = [0.32, 0.45, -0.12, 0.67]

These vector values are illustrative.

Step 4: Self-Attention

The model generates Query, Key, and Value vectors from the token representations.

Q = XWQ

K = XWK

V = XWV

The attention mechanism identifies relationships between words.

Examples:

1. "Generative" relates to the type of AI.
2. "AI" relates to the action "creates."
3. "creates" relates to "new content."
4. "new" modifies "content."

Attention combines relevant information from different tokens.

Step 5: Contextual Representation

The attention outputs are processed through feed-forward layers to create contextual representations.

For example, the word "creates" is interpreted in relation to "AI" and "content," rather than being processed entirely in isolation.

These representations help the model understand relationships between words.

Step 6: Output Prediction

The output projection produces scores for possible next tokens. These scores are converted into probabilities, and the model selects a token according to its decoding strategy.

For example, given the input "Generative AI creates new," the model might predict "content" as the next token.

The process can continue until the model generates the required output.

Role of Attention:

1. Attention allows tokens to use relevant information from other tokens.
2. It helps represent relationships between subjects, actions, and objects.
3. It combines contextual information from the sentence.
4. It helps generate context-aware representations for token prediction.

Conclusion: A Transformer converts text into token IDs and embeddings, applies self-attention and feed-forward operations, and uses contextual representations to predict output tokens.